

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[module-4.1]

DISTRIBUTED COMPUTING - INTRODUCTION

TEXTBOOK REFERENCE

- This slides contain notes taken from
“Elements of Distributed Computing”, Vijay K. Garg
 - chapter 1, 2, 3

DISTRIBUTED SYSTEMS

*Distributed Systems: You know you have
(un sistema distribuito)
one, when the crash of a computer you've
never heard of, stops you from getting any
work done.*

— Leslie Lamport

DISTRIBUTED SYSTEMS

- Distributed Systems
 - computers that contain multiple processors connected by a communication network
- Why distributed systems (vs parallel systems)
 - scalability
 - modularity and heterogeneity
 - data sharing
 - resource sharing
 - geographical structure
 - reliability
 - low cost
- Why not only distributed systems (vs. parallel systems)
 - efficiency
 - updating memory is still faster than sending a msg

KEY CHARACTERISTICS & CHALLENGES

non posso dire, è stato fatto prima un'operazione su un computer di un'altra operazione su un altro computer (anche se prendo il timestamp, esso viene calcolato utilizzando il clock corrente del computer in cui è stato calcolato)

Absence of a shared clock

(non possiamo ordinare gli eventi in modo totale perché ogni computer ha il proprio clock: quindi, non posso ordinare/mettere in relazione a priori il tempo di eventi che avvengono su processi diversi)

- it is impossible to sync clocks of different processors due to uncertainty in communication delay

(non è possibile a livello teorico: velocità della luce finito)

- physical clocks cannot be used for sync

Absence of a shared memory

(se si vuole costruire memoria condivisa, dobbiamo costruirla come servizio, come middleware: sistema che si posiziona sopra il OS, in un contesto distribuito, e fornisce servizi alle applicazioni per fare cose (ad esempio, la comunicazione tra applicazioni)). Il middleware è indipendente dalle applicazioni che usufruiscono del servizio del middleware

- it is impossible for one processor to know the global state of the system
- it is difficult to observe any global prop

Absence of an accurate failure detection

- in *async* distributed systems (=no upper bound on msg communication time) it is impossible to distinguish between a slow processor or failed processor

è complicato capire quando un nodo va giù: è davvero andato giù oppure c'è traffico di rete (quanto sono disposto ad aspettare una risposta? timeout)? Capisco che è andato giù quando ci prova a comunicare, ma devo definire dei timeout entro cui mi aspetto una risposta

CHALLENGES: CONSISTENCY VS. AVAILABILITY

- Consistency: Garantisce che tutti i nodi vedano gli stessi dati nello stesso momento; equivale ad avere una singola copia aggiornata dei dati accessibile a tutti.
- Availability: Garantisce che ogni richiesta non fallita riceva una risposta (ad esempio per gli aggiornamenti), senza la garanzia che contenga i dati più recenti.
- Partition Tolerance: Il sistema continua a funzionare nonostante interruzioni o perdite di messaggi nella rete che dividono i nodi.

• CAP Theorem [Brewer 2000]

- any networked shared-data system can have at most two of three desirable properties:
 - consistency (C) equivalent to having a single up-to-date copy of the data;
 - high availability (A) of that data (for updates); and
 - tolerance to network partitions (P).
- Simple example - consider 2 nodes on opposite sides of a partition
 - ^{consentire ad almeno} ~~allowing at least~~ one node to update state will cause the nodes to become inconsistent, thus forfeiting C
 - likewise, if the choice is to preserve consistency, one side of the partition must act as if it is unavailable, thus forfeiting A.
 - only when nodes communicate it is possible to preserve both consistency and availability, thereby forfeiting P

Per evitare dati incoerenti, il sistema deve bloccare gli aggiornamenti su uno dei lati della partizione finché la rete non viene ripristinata. In questo modo il nodo agirà come se non fosse disponibile, rinunciando alla Disponibilità (A).

Permetti a uno o entrambi i nodi di accettare aggiornamenti nello stato. Tuttavia, poiché non possono comunicare per sincronizzarsi, i nodi diventeranno incoerenti tra loro, rinunciando alla Consistenza (C).

È possibile solo quando i nodi riescono a comunicare normalmente, il che significa presupporre l'assenza di partizioni di rete (rinunciando quindi alla Tolleranza alle partizioni, P).

(E. Brewer, "CAP Twelve Years Later: How the "Rules" Have Changed", IEEE Computer, Feb 2012)

(E. Brewer, "Towards Robust Distributed Systems," Proc. 19th Ann. ACM Symp. Principles of Distributed Computing (PODC 00), ACM)

LATENCIES

Come faccio a distinguere tra latenza ed un crash (fallimento)? Non è possibile. Introduzione dei timeout (se non mi arriva la risposta entro un certo t, ASSUMO che ci sia stato un crash del nodo)

- In its classic interpretation, the CAP theorem ignores latency, although in practice, *latency and partitions are deeply related*
- Operationally, the essence of CAP takes place *during a timeout*, a period when the program must make a fundamental decision - *the partition decision*:
 - cancel the operation and thus decrease availability, or
 - proceed with the operation and thus risk inconsistency
- Thus, pragmatically, **a partition is a time bound on communication**
 - failing to achieve consistency within the time bound implies a partition and thus a choice between C and A for this operation
- These concepts capture the core design issue with regard to latency: *are two sides moving forward without communication?*

un limite temporale alla

ogni volta che c'è un ritardo di rete (latenza) o un timeout, il sistema è costretto a decidere se continuare a lavorare "al buio" oppure fermarsi in attesa dell'altro nodo.

Quando scatta il timeout, il programma si trova di fronte a un bivio obbligato: Opzione A: Continuare senza comunicare (Privilegiare l'Availability): Il nodo decide di procedere, accetta la richiesta dell'utente e aggiorna il proprio stato locale. Rischio: Poiché l'altra parte del sistema non sa cosa sta succedendo, i dati sui due nodi divergeranno (rischio di Inconsistenza / perdita di C).

(E. Brewer, "CAP Twelve Years Later: How the "Rules" Have Changed", IEEE Computer, Feb 2012)
Opzione B: Annullare l'operazione (Privilegiare la Consistency): Il nodo decide di rifiutare la richiesta o di bloccarsi finché la comunicazione non viene ripristinata. Rischio: L'utente si vede rifiutare il servizio (perdita di Availability / A).

"CANONICAL" DISTRIBUTED SYSTEMS PROGRAMMING MODEL

Qual'è il paradigma più opportuno per programmare un sistema distribuito? "più opportuno" significa che dal punto di vista del Designer e Sviluppatore, possa essere al giusto livello di astrazione per costruire programmi da mettere in esecuzione nel contesto distribuito

- Basic programming model
 - applications as sets of heavyweight processes that communicate by exchanging messages through channels
 - each process can have multiple threads, communicating through shared memory

- Models based on "traditional" programming paradigms

- examples

- procedural

Rimanendo nel paradigma procedurale (C), aggiungo un layer (un middleware) che mi permette di chiamare la procedura, da dove voglio, di un specifico nodo di un sistema distribuito

- remote procedure call

- object-oriented

Da procedurale, facciamo la stessa cosa nel Object oriented

- distributed object computing

- remote method invocation

Io ho parti di programma in diversi nodi, composti da oggetti, e voglio metterci un middleware per permettere ad un oggetto di chiamare un metodo di un oggetto su un altro nodo

- implemented by middleware

- e.g. CORBA, Java RMI

una delle caratteristiche interessanti è interoperabilità (parti di OOP distribuite con linguaggi diversi che comunicano, tramite invocazioni di metodo, attraverso il middleware CORBA)

Ci permette di astrarre dal fatto che sotto abbiamo un sistema distribuito, rendendo tutto trasparente (posso astrarre dai dettagli di rete), permettendo automaticamente di scalare da un contesto locale ad uno distribuito, grazie ad un middleware

LIMITS OF TRADITIONAL PARADIGMS

- Distributed computing ~~call for dealing with aspects~~ ^{richiede di affrontare aspetti} that cannot be *abstracted away*
 - the idea of applying traditional OOP to distributed computing abstracting from distribution (*transparency*) does not work
 - see “*A Note on Distributed Computing*” by Jim Waldo et al (1994)
- Beyond traditional paradigms
 - towards approaches based on decentralisation of control & asynchronous interaction models
 - (asynchronous) message passing models
 - actor paradigm applied to distributed systems
 - message-oriented middleware (MOM)

L'articolo citato nella slide dimostra che questo intento è un'illusione concettuale per due ragioni fondamentali:

- Gestione dei fallimenti (Natura delle eccezioni): In locale un metodo o funziona o fallisce per un bug algoritmico; in rete subentrano fallimenti parziali (cavi staccati, nodi bloccati, timeout). Mascherare una chiamata remota come locale costringe l'applicazione a gestire eccezioni di rete ovunque, rompendo l'astrazione.
- Latenza e Principio di Località: Una chiamata locale impiega nanosecondi, una remota millisecondi (100.000 volte più lenta). Ignorare la posizione fisica degli oggetti porta a progettare software con prestazioni disastrose.

Si tratta di infrastrutture/software di rete che si frappongono tra i processi per gestire l'invio e la ricezione di messaggi in modo affidabile. Offrono disaccoppiamento temporale e spaziale

Poiché l'interazione è asincrona per natura, il modello si adatta perfettamente alla rete: non c'è alcuna differenza concettuale tra inviare un messaggio a un attore sulla stessa macchina o a un attore su un server remoto.

Nei sistemi distribuiti è necessario un modello decentralizzato: ogni nodo opera in autonomia sulla propria memoria locale e coordina le attività con gli altri senza che esista una "regia centralizzata" bloccante.

FROM AN ARCHITECTURE & SW ENGINEERING PERSPECTIVE

- Service-oriented architectures & computing
 - services, microservices
 - relationship with domain-driven design
- Event-driven architectures
 - event-driven microservices
 - stream-based architectures
- Cloud-based architectures & computing
 - large-scale systems, scalability

ASPECTS CONSIDERED HERE

- Models of distributed computation
 - events
- Some main distributed algorithms
 - coordination, observation, control

MODELING DISTRIBUTED COMPUTATIONS

Modello delle computazioni distribuite

Distributed computation model

– loosely coupled set of processes

sending messages through unidirectional channels

Proprietà e Garanzie del Canale di Comunicazione

- infinite buffer, error free
- no assumption on the ordering of the messages
- any message sent => arbitrary but finite delay

Non si assume che il canale rispetti l'ordine d'invio. Se P1 invia m1 e poi m2, P2 potrebbe ricevere m2 prima di m1.

Ogni messaggio inviato arriverà prima o poi a destinazione (il ritardo è finito), ma non possiamo fare previsioni su quanto tempo impiegherà (il ritardo è arbitrario).

– state of the channel

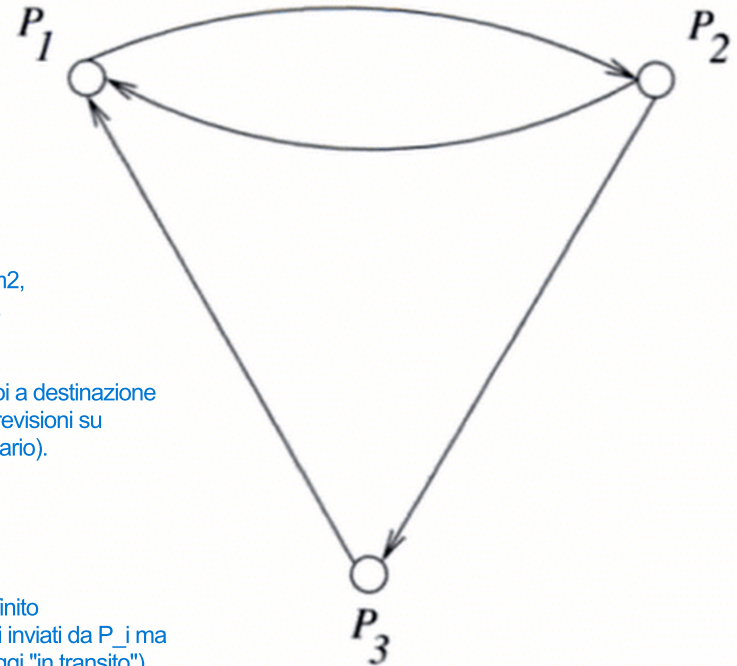
- sequence of messages sent not yet received

In qualsiasi istante, lo stato del canale C_{ij} è definito dall'insieme/sequenza di messaggi che sono stati inviati da P_i ma che non sono ancora stati ricevuti da P_j (messaggi "in transito").

Assumption

– asynchronous distributed systems

I processi non condividono memoria o un orologio fisico centrale; comunicano esclusivamente tramite lo scambio di messaggi.



Un clock è un meccanismo che assegna un'etichetta oraria (timestamp) a ciascun evento, fornendo un criterio per ordinare le azioni all'interno di un sistema.

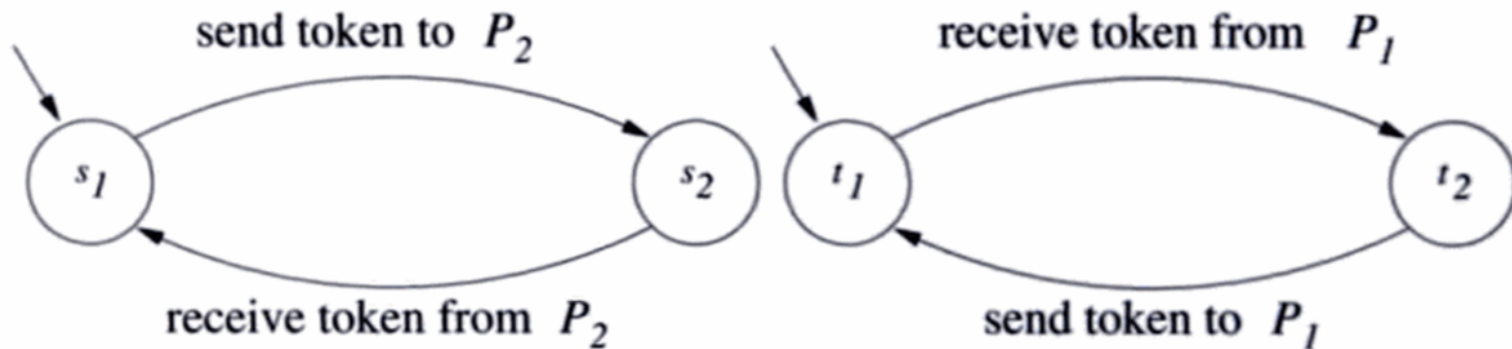
PROCESSES: STATES, EVENTS

- Process model

Un processo P_i è modellato formalmente da tre elementi:

- set of states, initial condition and a set of events
- each event may change state of the process and the state of 1 channel incident to that process

- The behaviour of each process can be described visually with state transition diagrams



Esistono tre tipi principali di eventi per un processo P_i :

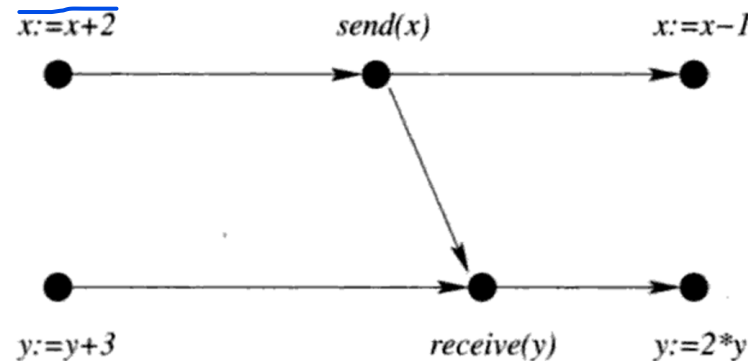
- Evento Locale: Modifica solo lo stato interno del processo P_i (es. la modifica di una variabile in memoria o un calcolo). Non altera lo stato di alcun canale.
- Evento di Invio (Send event): Modifica lo stato interno del processo (es. incrementa il clock locale). Modifica lo stato di 1 canale uscente (aggiunge il messaggio inviato alla coda del canale $C_{\{ij\}}$).
- Evento di Ricezione (Receive event): Modifica lo stato interno del processo (es. aggiorna le variabili locali con i dati del messaggio o applica la regola del max del Vector Clock). Modifica lo stato di 1 canale entrante (rimuove il messaggio ricevuto dalla coda del canale $C_{\{ji\}}$).

Nota importante: Un singolo evento agisce sempre su un solo canale ad esso incidente (entrante o uscente), mai su più canali contemporaneamente.

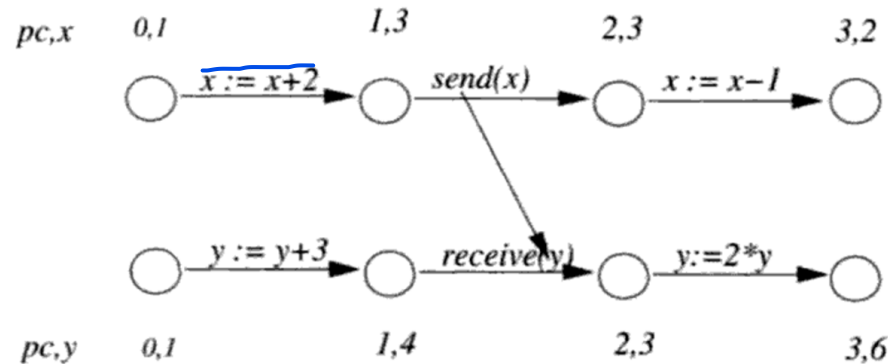
EVENT VS. STATE MODELS

- Depending on the applications, it could be that an application is modelled more naturally in terms of events, not states

event
diagram



state
diagram



MAIN MODELS

in un sistema distribuito

è un buon modello fino ad un certo punto: non catturo le failure (presupponiamo che non ci sono failure)

Interleaving model of computation

(State diagram o Petri net per rappresentare questo modello con un esecutore globale che può seguire tutte le strade possibili)

Sono in grado di ordinare tutti gli eventi (con Happened-before, riesco ad ordinare solo quelli che hanno relazione causa-effetto)

- assuming a total order among the events

Happened-before model

Rappresento ogni azione come un evento che può avvenire (non vengono più rappresentati gli stati di un sistema) (gli eventi come dei nodi e le relazioni Happened-before come degli archi: si tratta di un grafo)

(Qui, happened-before, dice che un evento avviene prima di un altro se uno è la causa dell'altro che è un effetto)

- assuming partial orders, with a total order for events related to the same process

(Qui, happened-before, uso il clock fisico per dire chi avviene prima e chi dopo)

Potential causality model

- no assumption even for single process composed by more than one thread

Significa che il Potential Causality Model non fa alcuna assunzione precostituita sull'ordine degli eventi locali, neanche all'interno dello stesso processo. A differenza del modello classico, non obbliga due eventi ad essere legati solo perché si trovano dentro lo stesso processo P1:

Non assume l'ordine sequenziale a prescindere: Non applica ciecamente la regola "se e1 viene prima di e3 su P1, allora e1 → e3".

Traccia la vera causalità tra thread: Se il Thread A (\$e_1\$) e il Thread B (\$e_3\$) non interagiscono, il modello li considera indipendenti, anche se risiedono nello stesso processo. Se il Thread A invia un dato al Thread B tramite una coda di messaggi o una variabile condivisa, solo allora il modello traccia una freccia causale tra i due thread.

Nel modello Happened-Before tradizionale di Lamport, l'assunzione di base è che un processo sia un'entità monolitica e single-threaded, dove tutti gli eventi eseguiti su quel nodo avvengono in una sequenza strettamente lineare (ordine totale locale).

Cosa fa di diverso dall'Interleaving? L'Interleaving forza una sequenza totale (fittizia) di tutti gli eventi. Il Happened-Before modella la realtà fisica del sistema distribuito: ordina solo gli eventi che sono causalmente dipendenti tra loro e lascia non ordinati (concorrenti) quelli indipendenti. È il modello di riferimento per osservare il comportamento dei programmi distribuiti e monitorarne le proprietà tramite i Logical Clock (orologi di Lamport) o i Vector Clock.

Nel modello a interleaving si ipotizza l'esistenza di un ordine totale globale tra tutti gli eventi del sistema, come se si susseguissero lungo una sola linea temporale fisica. Nella realtà distribuita questo ordine assoluto non esiste a causa dell'assenza di un orologio condiviso; l'interleaving non descrive l'unica sequenza reale trascurando la rete, ma viene utilizzato per catturare ed esaminare teoricamente tutti i possibili scenari di esecuzione (tutte le combinazioni valide di eventi), pur consapevoli di non poter stabilire a posteriori quale di essi si sia effettivamente verificato sul campo (In altre parole, siccome nei sistemi distribuiti non esiste un orologio unico condiviso per misurare il tempo fisico esatto tra nodi diversi, non possiamo dire con certezza "l'evento A sul Server 1 è avvenuto un millisecondo prima dell'evento B sul Server 2").

HAPPENED BEFORE MODEL

Esempio: invio di un messaggio da un computer avviene prima della ricezione del messaggio da un'altro computer (quindi, invio è la causa, ricezione è l'effetto e quindi la ricezione e gli eventi dopo la ricezione avvengono dopo dell'invio e di tutti gli eventi accaduti nel computer che ha mandato il messaggio)

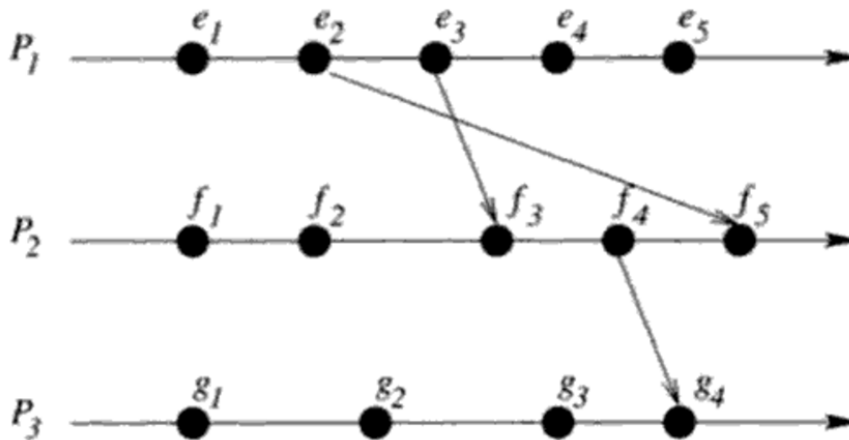
- **Lamport observation:**
 - ~~in a true distributed system only partial orders among events can be defined, using the *happened before* relation~~
 - in un vero sistema distribuito è possibile definire soltanto ordini parziali tra gli eventi utilizzando
- **Model**
 - process $P_i \Rightarrow$ generates a sequence of local states and events:
 $S_{i,0} \ e_{i,1} \ S_{i,1} \ e_{i,1} \ \dots \ e_{i,l-1} \ S_{i,l}$
 - The *happened before* relation $\rightarrow$ is the smallest relation that satisfies:
 - $(e \preceq f) \text{ or } (e \rightsquigarrow f) \Rightarrow e \rightarrow f$
 - if $\exists g: (e \rightarrow g) \text{ and } (g \rightarrow f) \Rightarrow (e \rightarrow f)$
 - where
 - $\preceq$ = *immediately-precedes* relation, defined between events of the same process
 - $e \preceq f$ if e immediately precedes f in the sequence of events at P_i
 - $\rightsquigarrow$ = *remotely-precedes* relation
 - $e \rightsquigarrow f$ (where e is at P_i e f is at P_j) if e is the send event of a message and f the receive event of the same message

Happened-before è la relazione che cattura l'idea di ordinamento parziale causale di una specifica esecuzione distribuita, ordinando solo gli eventi legati da causa-effetto e lasciando non confrontabili (concorrenti) tutti gli eventi indipendenti. Logical e Vector clock sono le implementazioni concrete (o algoritmi) per rappresentare la relazione Happened-Before all'interno del sistema.

RUN IN A HAPPENED-BEFORE MODEL

- A *run* or *computation* in a happened before model is defined as a tuple $(E, \rightarrow)$ where E is the set of all events and $\rightarrow$ is a partial order on events in E ^{tale} such ^{che} that all events within a single process are totally ordered.

Una catena di causa-effetto (che nel diagramma del Happened-before model corrisponde a un cammino orientato tra due nodi/eventi) può essere composta sia da un solo passo (relazione diretta) sia da più passi (relazione indiretta, grazie alla proprietà di transitività).



Quando consideri eventi che avvengono su processi diversi, puoi ordinarli solo se sono collegati da una catena di causa-effetto (un messaggio inviato e ricevuto, diretto o indiretto).

happened-before diagram

- Un Ordine Totale (Total Order): È una relazione in cui ogni singola coppia di elementi può essere confrontata.
- Un Ordine Parziale (Partial Order): È una relazione in cui SOLO ALCUNE coppie di elementi sono confrontabili, mentre altre NON lo sono.

Significa semplicemente che sono casualmente indipendenti: Ciò che è successo in e non ha potuto in alcun modo influenzare ciò che è successo in f , e viceversa.

- $e \rightarrow f$ iff it contains a directed path from the event e to f
- If two events are not related by the $\rightarrow$ then they ^{are} **concurrent**:
 $e \parallel f = \text{not } (e \rightarrow f) \text{ and not } (f \rightarrow e)$

Se tra l'evento e e l'evento f non esiste alcuna catena di causa-effetto (nessun messaggio scambiato tra i processi che li collegi, né direttamente né tramite altri nodi intermedi), allora il sistema non può e non vuole stabilire chi dei due sia venuto prima.

POTENTIAL CAUSALITY MODEL

- Happened before $\Rightarrow$ total order among events inside the same process.
Nel modello Happened-Before di Lamport: $e \rightarrow f$ (vengono trattati come legati da causa-effetto solo perché accadono sullo stesso nodo in sequenza).
 - however it is not true that all of these events have *cause/effect* relationship
All'interno di uno stesso processo, non tutti gli eventi in sequenza hanno una relazione di causa-effetto reale. La vera causalità interna a un processo è un ordine parziale, non totale.
 - *causality* relation is a partial order between events within the same process.
 - Often difficult or expensive to determine, then we consider the *potential causality relation*
significa che tracciare la causalità reale perfetta richiede troppe risorse; pertanto ci si accontenta della causalità potenziale, che rimuove i finti legami d'ordine tra eventi locali indipendenti mantenendo solo le dipendenze che potrebbero essere davvero legate da causa ed effetto.
- Potential *causality relation* $\rightarrow^p$ on the event set = smallest relation satisfying:
Si assume che un evento e possa potenzialmente influenzare un evento f sullo stesso processo se c'è una possibilità che l'informazione o lo stato del processo sia fluìto da e a f
 - if an event e causa potenzialmente potentially causes another event f on the same process, then $e \rightarrow^p f$
 - if e is a send of a message, and f the corresponding receive, then $e \rightarrow^p f$
 - if $e \rightarrow^p f$ and $f \rightarrow^p g$, then $e \rightarrow^p g$
- If two events are not related by the $\rightarrow^p$ then they are called *independent*
Se due eventi e ed f non sono legati da $\rightarrow^p$ in nessuno dei due versi, si dicono indipendenti (cioè concorrenti).

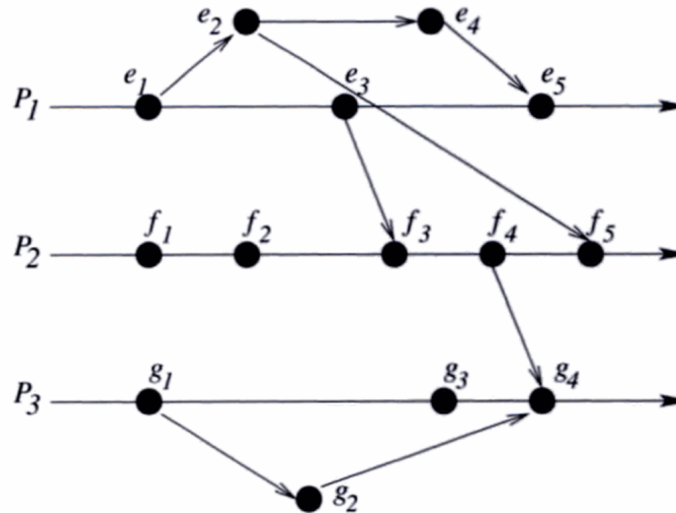
EXAMPLES

- A process receives 2 messages from different ports and updates different objects based on these 2 messages
 - the receives of these messages do not potentially cause each other, although one may have happened before the other
- A process based on 2 threads that access mutually disjoint sets of objects

POTENTIAL CAUSALITY DIAGRAM

- A potential causality diagram is defined as a partially ordered set $(E, \rightarrow^p)$ where E is the set of all events in the system and $\rightarrow^p$ is the potential causality relation.

che traccia solo i legami di causa-effetto reali o potenziali tra gli eventi, senza forzare un ordine sugli eventi locali che sono del tutto indipendenti tra loro.



potential causality diagram

Un normale diagramma Happened-Before (ovvero un'esecuzione reale in cui tutti gli eventi dello stesso processo sono totalmente ordinati) è coerente (consistent) con il diagramma di causalità potenziale se la relazione $\rightarrow^p$ è un sottoinsieme di $\rightarrow$.

Un diagramma Happened-Before è coerente con un diagramma di Causalità Potenziale se conserva tutte le dipendenze causali reali ($\rightarrow_P$) e si limita ad aggiungere un ordine sequenziale tra gli eventi locali indipendenti.



Potential Causality diagram

Happened-before diagram

- Given a $(E, \rightarrow^p)$, a $(E, \rightarrow)$ is *consistent* with it if $\rightarrow^p \subseteq \rightarrow$
- A potential causality diagram is equivalent to the set of happened before diagrams that are consistent with it

MODELS COMPARISON

Un software distribuito può produrre diverse esecuzioni a seconda degli input, della latenza di rete e dei tempi di esecuzione. Ciascuna esecuzione reale genera un Diagramma di Causalità Potenziale ($E, - >_p$), che traccia solo i veri vincoli di causa-effetto necessari affinché il programma funzioni correttamente.

- These models can be used for capturing the behaviour of a distributed program

Un diagramma Happened-Before è un ordine parziale. Se due eventi su processi diversi sono concorrenti ($e1 \parallel f1$), un osservatore esterno con un tempo assoluto ipotetico potrebbe vederli accadere nell'ordine ($e1, f1$) oppure ($f1, e1$). Il diagramma Happened-Before equivale quindi a tutte le possibili interleaving globali che rispettano le frecce del grafo.

- a distributed program can be viewed as a set of potential causality diagrams that it can generate.
- a potential causality diagram in turn is equivalent to a set of happened before diagrams.
- each happened before is equivalent to a set of global sequences of events (or states)

Model	Basis	Type
Interleaving	Physical time	Total order on all events
Happened before	Logical order	Total order on one process
Potential causality	Causality	Partial order on a process

- Which of these models is appropriate depends on the application for which the model is being used

EXAMPLES

- Verification of distributed program  Verifica Formale (Interleaving Model)
Scopo: Dimostrare matematicamente che un algoritmo è corretto (es. "non andrà mai in deadlock").
 - to prove certain properties, the interleaved is OK
-  Catturare il comportamento effettivo di un'esecuzione per verificare se una determinata proprietà si è verificata realmente. If we want to *capture the behaviour* and check if some property became true in it (we have to observe it), then the appropriate model is happened before
 - because a global sequence cannot be observed
- Distributed debugging  Analizzare il sistema per scoprire se una certa proprietà globale avrebbe potuto verificarsi in uno qualsiasi dei comportamenti validi del programma (es. individuare race condition o stati di errore nascosti).
 - we want to capture the behaviour, asking whether some global property *could have* become true in a behaviour of that program, then it is our advantage to capture only partial orders corresponding to potential causality

FROM MODEL TO MECHANISMS

- Mechanisms to implement the happened-before model
 - Logical Clocks
 - Vector Clocks

LOGICAL CLOCKS

Ogni processo ha il suo contatore (Logical Clock)

- **Logical clocks** (Lamport 1978)

Un orologio logico attribuisce a ciascun evento un valore numerico $C(e)$ (un timestamp). Il suo obiettivo primario è garantire la coerenza causale: Se un evento a ha causato un evento b , l'orologio logico assicura che $C(a) < C(b)$.

- **tracking the happened before relation**

- **giving a total order that could have happened instead of the total order that did happened.**

tenere traccia della

Fornire

che sarebbe potuto accadere al posto dell'ordine totale

che è effettivamente accaduto.

Significa che l'orologio logico costruisce un ordine totale arbitrario ma valido, anziché tentare di scoprire l'ordine reale del tempo fisico.

- **Def:**

- A logical clock C is a function of each process P_i assigning a number $C_i(a)$ to any event a of the process such that the Clock Condition holds:

associata a ciascun

che assegna un

a qualsiasi

di quel processo

in modo tale che

sia soddisfatta

$$\forall a \in P_i, b \in P_j : \text{if } a \rightarrow b \text{ then } C_i(a) < C_j(b)$$

- It is easy to see that the Clock Condition is satisfied if the following two conditions holds:

verificare che

valgono

- if $a, b \in P_i$ and $a \rightarrow b$, then $C_i(a) < C_i(b)$
 - if $a = \text{send}(m)$ by P_i and $b = \text{receipt of } m \in P_j$ then $C_i(a) < C_j(b)$

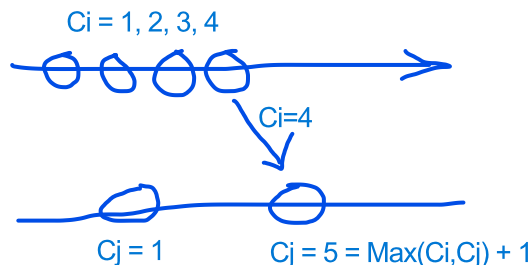
(all'interno dello stesso processo, l'evento successivo ha un numero maggiore).

→ (il numero della ricezione di un messaggio è sempre strettamente maggiore del numero del suo invio).

Leslie Lamport. 1978. *Time, clocks, and the ordering of events in a distributed system*. Commun. ACM 21, 7 (July 1978), 558-565. DOI=<http://dx.doi.org/10.1145/359545.359563>

IMPLEMENTATION

- Using a counter C for each process
- Implementing the two rules
 - (1) Each process increments the counter C between ^{qualsiasi} ~~any~~ 2 successive events
 - (2) If event a is the *sending* of a message m by process P_i , then the message m contains a timestamp $T_m = C_i(a)$.
(piggybacking: al messaggio metto anche il contatore del processo P_i)
 - (3) Upon receiving a message m with T_m , process P_j sets C_j to $\max(C_j, T_m)$ (and then increment the counter according to rule (1))



LOGICAL CLOCKS: LIMITATIONS

- With logical clocks

$$\forall a, b : a \rightarrow b \Rightarrow C(a) < C(b)$$

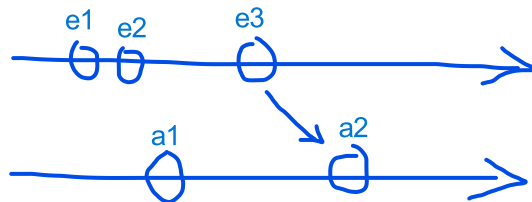
Se sappiamo già che l'evento a ha causato l'evento b, allora siamo certi che il valore dell'orologio logico di a sarà minore di quello di b ($C(a) < C(b)$).

but not viceversa, that is it is **not** true that:

$$\forall a, b : C(a) < C(b) \Rightarrow \cancel{a} \rightarrow \cancel{b}$$

Se guardiamo solo i timestamp numeri interi assegnati da Lamport e vediamo che $C(a) = 2$ e $C(b) = 5$, non possiamo sapere se a ha davvero causato b oppure se a e b sono due eventi totalmente concorrenti/indipendenti.

- An approach that overcomes this limitation: **vector clocks**
(Fidge 1988)(Mattern 1988)



Colin J. Fidge (February 1988). "Timestamps in Message-Passing Systems That Preserve the Partial Ordering"(PDF). In K. Raymond (ed.). *Proc. of the 11th Australian Computer Science Conference (ACSC'88)*. pp. 56–66

Mattern, F. (October 1988), "Virtual Time and Global States of Distributed Systems", in Cosnard, M. (ed.), *Proc. Workshop on Parallel and Distributed Algorithms*, Chateau de Bonas, France: Elsevier, pp. 215–226

VECTOR CLOCKS

Il Vector Clock riesce a distinguere i casi concorrenti perché quando ogni componente del vettore è minore o uguale dell'altro allora sono sicuro che l'evento "happened-before" dell'altro

→ Tenere traccia della conoscenza che ha ogni processo sugli altri processi (conoscenza del proprio clock e degli altri)

- A vector clock is a function assigning a vector VC of size k to each event a that: $\forall a, b: a \rightarrow b \Leftrightarrow VC(a) < VC(b)$

where given 2 vectors v, w

- $v < w$ means $\forall k \in [1..N], v[k] < w[k]$ and $\exists j \in [1..N] \mid v[j] < w[j]$
- $v \leq w$ means $(v < w) \text{ opp } (v = w)$

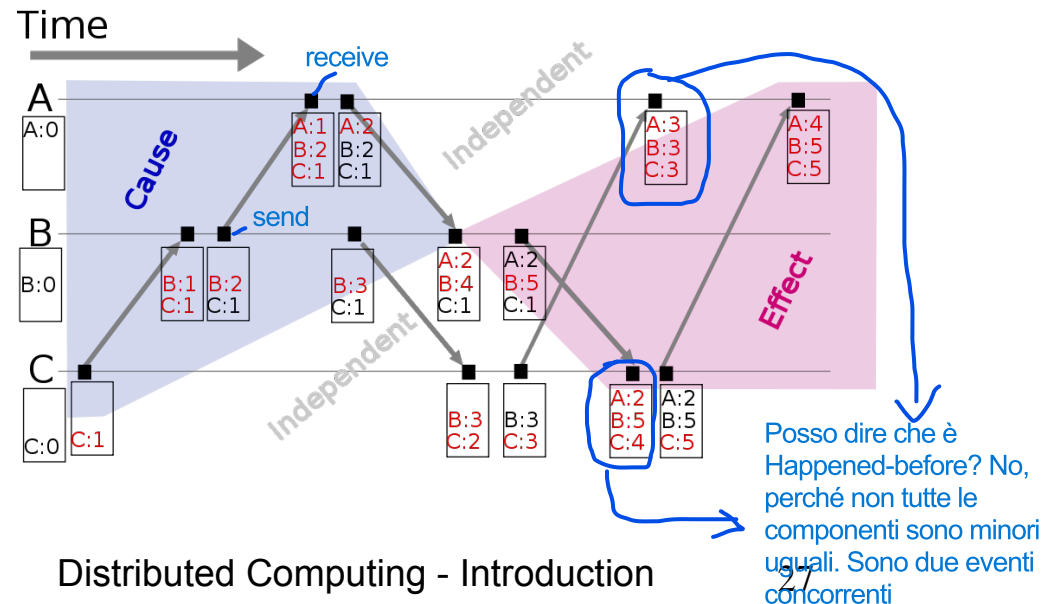
Un vettore v è minore o uguale a w se è strettamente minore ($v < w$, come definito sopra) oppure se è esattamente identico in ogni posizione ($v = w$, cioè $v[k] = w[k]$ per tutti gli indici k).

Limite: numero dinamico dei processi (non conosco il numero di processi che aumenta/diminuisce nel tempo). Nella ricerca ci sono molte variazioni di esso

- Example
 - 3 processes A, B, C

Un vettore v è considerato strettamente minore di un vettore w ($v < w$) se e solo se valgono entrambe queste condizioni:
Non è mai superiore in nessuna posizione: Per ogni indice k da 1 a N , l'elemento $v[k]$ è minore o uguale a $w[k]$ ($v[k] \leq w[k]$).
È strettamente minore in almeno una posizione: Esiste almeno un indice j per cui $v[j] < w[j]$.

Quando capita un evento, incremento il mio contatore. Quando c'è una send (metto il contatore in piggybacking di chi fa la send e della sua conoscenza degli altri contatori), aggiorno il mio contatore con il Max e la conoscenza degli altri contatori



IMPLEMENTATION

- Using a vector V of counters for each process P (vector size = number of processes)
 - Each process P_i increments the counter $V[i]$ between any 2 successive events
 - If event a is the *sending* of a message m by process P_i , then the message m contains the vector clock $V = V_i(a)$ (as a timestamp).
 - ~~Upon receiving~~ ^{Alla ricezione di} a message m with a vector clock V , process P_j sets each $V_j[i]$ to be $\max(V_j[i], V[i])$, for each element i .